A Layered Approach in Graph Intelligence with Contrastive Semantics for Network Intrusion Detection

Aditya S[1], Vignesh Karthikeyan[1], Vedansh Kumar[1], Sandosh S[1*]

[1*]School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India.

*Corresponding author(s). E-mail(s): sandosh.s@vit.ac.in;
Contributing authors: aditya.s2022a@vitstudent.ac.in;
vignesh.karthiekyan2022@vitstudent.ac.in;
vedansh.kumar2022@vitstudent.ac.in;

Abstract

Recent Network Intrusion Detection Systems (NIDS) struggle with such dynamism as the concept drift, as well as limited labelled data. Graph-based methods, such as LAGER, can realise a changing network topology and can work online, but can only use the statistical characteristics of the network and provide minimal semantic insights. On the other hand, semantic encoders like PacketCLIP match traffic and natural language descriptions, supporting interpretability, but they are not adaptable online. This paper proposes a unified system that combines online graph dynamics and semantic embeddings to achieve adaptive and interpretable intrusion detection. The system utilises layer-wise graph processing with contrastive semantic vectors, enhanced by real-time drift adaptation mechanisms and cross-attention modules, to combine graph and semantic features. Real-world experimentation demonstrates competitive accuracy, enhanced interpretability, and flexibility compared to existing techniques, making it applicable to dynamically changing cybersecurity scenarios.

Keywords: Network Intrusion Detection, Graph Neural Networks, Contrastive Learning, Concept Drift, Semantic Embeddings, Multimodal Learning, Drift Adaptation

1 Introduction

Modern NIDS struggle with dynamic, real-world traffic where concept drift is constant and labels are scarce. Graph-based approaches (LAGER) capture evolving topologies and operate online, but rely only on statistical features, offering little semantic understanding. In contrast, semantic encoders (PacketCLIP) align traffic with natural language descriptions, enabling interpretability, but lack online adaptability. The challenge is to combine online graph dynamics with semantic embeddings to achieve adaptive and interpretable intrusion detection.

Building an online, semantically enriched NIDS involves tackling several open challenges:

High-volume semantic encoding: Network traffic is massive and noisy; generating compact, multimodal embeddings that preserve both protocol-level details and semantic meaning is computationally demanding.

Fusion of graph and semantic features: Statistical graph dynamics capture who communicates with whom, while semantic embeddings capture what the communication means. Designing a joint representation space without redundancy or overfitting is non-trivial.

Robust drift characterization: Real networks evolve continuously. The system must differentiate benign evolution (new devices, services) from adversarial drift (botnets, scans) while maintaining plasticity (adapt quickly) and stability (resist poisoning).

Scalability and efficiency: Real-time updates of large dynamic graphs combined with semantic inference introduce significant overhead. Solutions require graph sampling, embedding compression, and edge-optimized models.

Interpretability and trust: Analysts need clear explanations—not just which device is anomalous but also why it is anomalous, combining graph evidence with semantic labels for actionable insights.

The primary goal is to create an adaptive, interpretable NIDS that unifies graph-based online learning with semantic embeddings. Specifically, the objectives are to:

- Develop a unified framework where evolving traffic graphs are enriched with contrastive semantic vectors, combining topology and meaning for richer detection signals.
- Design drift-aware mechanisms that not only detect anomalies but also characterize drifts in real time as benign behavior or malicious activity.
- Evaluate extensively on real-world datasets, benchmarking against purely statistical and purely semantic baselines to measure accuracy, robustness, and interpretability.
- Introduce continual learning capabilities, enabling the system to incrementally adapt to new benign patterns without retraining from scratch.
- Optimize scalability for deployment, using techniques like graph sampling, embedding compression, and edge-friendly inference to handle enterprise-scale traffic.

2 Related Work

2.1 Multimodal Learning and Semantic Integration

Masukawa et al. [1] proposed PacketCLIP, a multi-modal model of combining a packet data representation with natural language semantics with contrastive pretraining and hierarchical GNN reasoning. The scheme is much more accurate in the classification of traffic, and offers human operators natural language explanations of the packet behavior, greatly increasing its understandability. Nevertheless, classical models are not able to comprehend the semantics of network behaviours and reasoning about the semantics of cyberattacks is one of the fundamental research problems.

2.2 Graph-Based Network Intrusion Detection

Shafi et al. [2] designed LGTBIDS (Layer-wise Graph Theory-Based Intrusion Detection System), a system that represents wireless communication networks in the form of graphs and conducts an independent layer-wise analysis to locate the attacked devices of Beyond 5G networks. The system has low computational complexity and aims at overcoming attacks related to spoofing and signal strength. Nevertheless, it is only based on metrics of energy conservation and secrecy rate, has no semantic awareness of attack patterns, simple statistical measures, and is limited to wireless networks.

Sun et al. [3] introduced GNN-IDS, which is a combination of attacks graphs with real-time measurements through GNNs (GCN, GAT). This methodology represents both topological and temporal patterns, which reduce the number of false positives in contrast to a conventional ML-based IDS. It is however based on pre-defined attack situations and manual assembly of attack graphs, is not adapted to new attack patterns, and does not provide semantic interpretation in addition to visualization.

He et al. [4] presented a new framework named LAGER (Layer-wise Graph Feature Extractor) which represents networks as layer-wise attributed graphs (TCP, UDP, ARP, ICMP) and learns representative embeddings using GNNs. The system proposes the Concept Drift Compatibility Index (CDCI) of assessing drift detection performance in NIDS, which overcomes the weaknesses of statistical feature extractors, which offer neighborhood-sensitive, protocol-sensitive representations. But it has no semantic knowledge of patterns identified - it simply gives anomaly scores based on statistical differences but it does not give an explanation of what anomalies mean - and it cannot distinguish benign concept drift and malicious activity beyond statistical differences, which leads to low interpretability even though it can visualize graphs.

Caville et al. [5] designed self-supervised graph neural network based NIDS in node embeddings and reconstruction loss, called Anomal-E, to detect anomalies in network flows. Being the pioneer in implementing self-supervised learning to intrusion detection on networks based on graph, it involves node2vec embeddings with the help of autoencoders reconstruction to determine anomalous network nodes. Nevertheless, it uses fixed network representations and does not take into account the temporal dynamics or concept drift, only works in offline evaluation, and does not adapt to changes in network patterns or differentiate the various types of concept drifts.

Lo et al. [6] proposed the E-GraphSAGE, a scalable version of the GraphSAGE framework on IoT network intrusion detection based on inductive learning of the graph representation through improved sampling methods. The methodology generalizes GraphSAGE to support large-scale IoT networks with a better neighbor sampling and aggregation algorithm, and it has a better performance on attack types peculiar to the IoT. Nevertheless, it only supports flow-based characteristics where completion of connection is necessary, so it is inappropriate in real-time detection mode, it does not have the means of concept drift, and limited semantic interpretation of binary classification outcomes.

2.3 Drift Detection and Adaptation Methods

Yoon et al. [7] introduced ARCUS (Adaptive Framework of Online Deep Anomaly Detection Under Complex Evolving Data Stream) that utilizes adaptive pooling of a group of autoencoders to control complex changing data streams. The system manages various concept drift (sudden, gradual, incremental, recurrent) in IoT and cloud data with an F1-score of more than 85%. However, it provides no semantic understanding and relies only on anomaly scores, operates reactively rather than proactively in handling drifts, and is limited to autoencoder architectures, reducing general applicability.

Gupta et al. [8] came up with NetGuard, a generative active adaptation framework to deal with concept drift and class imbalance at the same time. It uses the concept of density-aware active learning to select informative samples to be labeled and deep generative models (VAEs, RealTabFormer) to generate a variety of samples to be used in data augmentation to improve its F1-score by 0.60 to 0.86. But it is more oriented to the generative augmentation and active learning but is not supported with multi-modal cross-domain alignment of packet data and semantic descriptions, lacks graph-based reasoning and natural language understanding, and is constrained to conventional ML methods without the use of current cross-modal representation learning methods.

Diallo and Patras [9] proposed the ACID (Adaptive Clustering-based Intrusion Detection) that uses adaptive clustering along with neural embeddings to resource-constrained edge environments. The system proposes dynamic threshold adaptation, according to the network conditions, to ensure successful malicious traffic classification with fewer computational requirement. It is however based on a set of assumed clustering which might not be true in various network behaviors, cannot offer explanations with regards to the classification decisions and has a limited capacity to deal with new attacks without human modifications of the model.

The first, having been coined by Yang et al. [10] is named CADE (Concept Drift Adaptation through Deep Embedding), a framework whose main focus was on concept drift in network security domains. It relies on contrastive learning to generate consistent embeddings when the network is subject to varying conditions and proposes drain detectors distinguishing between benign and malicious concept changes. It however needs labeled data to work effectively on drift characterization, is developed as an offline classifier not an online detector and lacks ability to give natural language characterization of drifts detected and semantic understanding of network pattern changes.

2.4 Contrastive Learning for Network Security

Koukoulis et al. [11] introduced a self-supervised contrastive learning model using transformers that learned flow representations using raw packet sequences. The method shows the ability to detect anomalies with better performance and minimal labeled data, and is also able to adapt to traffic across different domains. Nevertheless, random packet segment augmentation may not be effective when the segments are overly similar or different and simplified tokenisation with a single token per packet may lose fine-grained information on individual bytes.

Sattar et al. [12] introduced a new framework ET-SSL (Encrypted Traffic Anomaly Detection using Self-supervised Contrastive Learning) that can identify anomalies in encrypted network traffic without labeled data and without payload analysis. It is accurate with low false positive rates, it is very fast and capable of processing traffic in real-time, and is effective in detecting the occurrence of zero-day attacks. But high-speed real-time deployment optimisation is required and the experiment is done based on simulated traffic as opposed to encrypted traffic in the real world.

Tan et al. [13] presented an extensive review of contrastive learning in NIDS, offering a review of the generic framework of advances, applications and architectures of contrastive learning to intrusion detection. This analysis shows that there are enhancements in detection accuracy, strength, and threat identification of the zero-day. Nevertheless, persistent issues comprise the imbalance in the data, the inability to detect rare attacks, the challenges with similar benign/malicious behavior, reliance on negative sampling choices, and excessive computation overhead requirements to use on a large scale.

Chen et al. [14] proposed a contrastive learning model named SimCLR, which relies on simple architectures (ResNet) and powerful data augmentation and nonlinear projection heads. It also shows state-of-the-art results in self-supervised and semi-supervised learning of visual representations, the composition of data augmentation, the design of the projection head, large batch sizes, and training steps are important. Nonetheless, it has high batch size and training duration, is dependent on powerful data augmentations, and still needs additional investigation to be able to generalize to new domains.

Zhang et al. [15] suggested CLE-TFE, which is a hybrid of supervised contrastive learning and GNNs to classify encrypted packets and flows simultaneously at the packet level as well as the flow level. The method proposed to augment the graph data at the traffic of the graph of bytes to have a powerful representation and thus can jointly train on multiple tasks with a small test on the cost. Nonetheless, trade-offs arise during the optimization of flow-level and packet-level activities together, and further research is necessary on learnable augmentations and hard negative mining.

2.5 Continual and Distributed Learning

Li et al. [16] suggested CLE-TFE, which is a hybrid of supervised contrastive learning and GNNs to classify encrypted packets and flows simultaneously at the packet level as well as the flow level. The method proposed to augment the graph data at the traffic of the graph of bytes to have a powerful representation and thus can jointly train on

multiple tasks with a small test on the cost. Nonetheless, trade-offs arise during the optimization of flow-level and packet-level activities together, and further research is necessary on learnable augmentations and hard negative mining.

2.6 Specialized Detection Systems

Zhai et al. [17] proposed the ODTC (Online Darknet Traffic Classification), which combines CNN and BiGRU to extract local space-time features and MLP to extract flow level abstract features. The system enhances feature extraction with the help of logistic chaotic mapping and utilizes the mechanism of multi-head attention, resulting in an increase in accuracy by 3-5% and a decrease in time and memory usage by 50%. However, it is focused on the darknet traffic and lacks generic network classification, semantic reasoning, adaptation to concept drift, and human-readable explanations.

To extract security properties as per the documentation Meng et al. [18] suggested NSPG, a NLP-based security property generator, using hardware-specific BERT (HS-BERT). The system significantly reduces manual work and achieves high accuracy on benchmarks. Nevertheless, some extracted properties are false positives or negatives, and its performance depends on the quality of documentation.

Lin et al. [19] designed ET-BERT, a method that modifies and fine-tunes a transformer pretraining to encrypted traffic with masked BURST modeling and same-origin BURST prediction. The method attains state-of-the-art results on encrypted traffic workloads and can be adjusted on small labeled data. However, its efficiency can decrease with the introduction of new protocols, and the model is susceptible to data poisoning.

Huoh et al. [20] proposed a flow-based GNN that utilizes geometry deep learning to project network flows on graphs. The end-to-end GNN model outperforms CNN and LSTM baselines and is flexible to input shape and multimodal. The performance can however be degraded when unseen encryption protocols are used, and more robustness should be achieved further.

Alwhbi et al. [21] surveyed the methods of machine learning and deep learning in the analysis and classification of encrypted traffic, giving a detailed review of the data collection, feature extraction, and model training. The analysis finds such issues as dynamic networking environments, high-dimensional data, minimal frameworks for large-scale deployment, susceptibility to obfuscation, partiality of sampling, insufficient generalization, and unrepresentative labeled datasets.

2.7 Explainability and Interpretability

Houssel et al. [22] compared GPT-4 and LLama3 LLMs to network intrusion detection, the first study to explore the feasibility of using LLMs as a NIDS classifier, but demonstrating that it is not viable, and excellent as explainability agents. Nevertheless, LLMs are low-performing primary detectors, have domain-sensitive attacks, inference latency is high, and they have hallucinations.

Neupane et al. [23] discussed explainable AI (XAI) comprehensively in intrusion detection systems, offering a generic X-IDS architecture and describing the issues of

research. The survey notes that there is no unanimous agreement on what explainability is, it is difficult to adapt explanations to different stakeholders, there are adequate standardized measures, and there are trade-offs between accuracy and interpretability.

Lee and Rew [24] made LIME integrate with Vision-Language Models (VLM) to identify CAN intrusion, producing global and local explanations that are readable to humans. The framework is highly interpretable and satisfactory to the users. Nevertheless, the real-time appropriateness of lightweight VLM models and optimized prompt engineering is constrained by computational overhead in resource-constrained settings.

El-Maghraby et al. [25] conducted a survey of deep packet inspection methods, including algorithms (Aho-Corasick, Wu-Manber), implementations, and applications. The analysis finds such challenges as bottlenecks in processing, problems with encrypted traffic, scaling problems, and the necessity of effective real-time solutions.

In summary, existing approaches either favor adaptability or interpretability, but rarely both, motivating the unified formulation presented in Section 3.

3 System Model and Problem Formulation

3.1 Research Gaps

In spite of the great improvements, there are still serious gaps. There has been a fundamental disjuncture: the current methods are either more adaptable or more interpretable but can hardly be both. Such systems as LAGER are highly adaptive but show no semantic explanations, whereas such systems as PacketCLIP are interpretable but do not have the mechanisms of constant adaptation. Complementary graph-based and semantic modalities are uncommon entities that have been successfully integrated in a unified framework, and this a valuable opportunity. No adequate emphasis is placed on the concept drift characterization - most drift-aware systems emphasize adaptation, and offer little information about what the drift is. Also, preserving semantic comprehension and interpretability in the context of adaptation is rarely addressed, and computational efficiency is insufficiently considered because advanced architectures such as transformer-based models and LLMs are highly efficient, but can render them unrealistic regarding processing a large network in real-time.

3.2 Problem Formulation

The subject of this work is the handling of online traffic with the help of graph-based representations to which semantic vectors are added, and the identification of drift as a benign or malicious one. The verification on real-world datasets, scalability analysis, and analyst interpretability are also within the scope. The main issue is that existing systems either identify drift in an unmeaningful way (statistical models), or identify meaning (semantic models) without adaptation. The solution should address both aspects.

4 Proposed Methodology

The proposed system brings together three core components: (i) a GATv2-based graph encoder that learns structural representations of network traffic, (ii) a QFormer cross-attention bridge that aligns graph embeddings with natural language semantics, and (iii) a drift-compensated multi-task classifier that performs adaptive and interpretable intrusion detection. Each component is described below, followed by a discussion of how they interact within the unified framework.

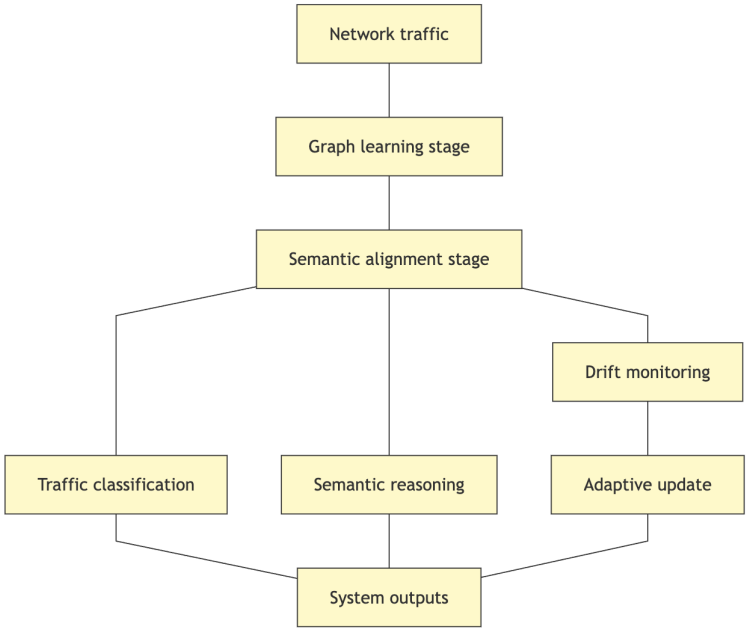

Fig. 1 System Architecture for Semantic and Drift-Aware Network Intrusion Detection.

4.1 Temporal Graph Construction

Network traffic is modelled as a sequence of temporal graph snapshots in order to preserve the structural and temporal properties of communication behaviour. Raw network flows from the CIC-IDS2017 dataset, comprising 3,119,345 flows across eight CSV files with 2,830,743 valid timestamped entries, are aggregated into fixed-length windows of 30 seconds. A sliding stride of 10 seconds is applied so that consecutive windows partially overlap, ensuring that attack activity near window boundaries is not missed. Each window is independently transformed into a graph snapshot, enabling

localised modelling of transient network behaviour. This process yields 4,874 temporal graph snapshots across the full dataset.

4.1.1 Graph Formation

For each temporal window, a directed graph $\mathcal{G}_t = (\mathcal{V}, \mathcal{E})$ is constructed. Nodes $\mathcal{V}$ represent unique IP addresses observed within the window, while edges $\mathcal{E}$ correspond to communication flows between IP pairs. Every edge carries the 77-dimensional CICFlowMeter feature vector associated with its underlying network flow, capturing packet-level statistics, protocol distributions, timing characteristics, and payload information. The graph-level attack label is determined by majority voting over all node labels, with non-benign classes prioritised so that a graph containing even a small proportion of attack traffic is labelled accordingly.

4.1.2 Node Feature Extraction

Each node $v \in \mathcal{V}$ is represented by a 77-dimensional feature vector obtained through mean aggregation of the CICFlowMeter features from its incident edges. Formally, the node feature vector is defined as

$$\mathbf{x}_v = \frac{1}{|\mathcal{N}_{\text{out}}(v)|} \sum_{e \in \mathcal{N}_{\text{out}}(v)} \mathbf{f}_e \in \mathbb{R}^{77}, \tag{1}$$

where $\mathbf{f}_e$ denotes the flow-level feature vector associated with edge e and $\mathcal{N}_{\text{out}}(v)$ is the set of outgoing edges from node v. When outgoing edges are unavailable, incoming edges are used as a fallback, and nodes with no incident edges receive a zero vector. The resulting features encode statistical characteristics such as flow duration, packet counts, byte volumes, inter-arrival times, flag distributions, and header lengths. All node features are globally normalised using a standard scaler fitted across the entire dataset.

4.1.3 Auxiliary Graph Properties

Beyond the primary node features, several auxiliary graph-level properties are computed for each snapshot. Community structure is identified using the Louvain algorithm applied to the undirected version of each graph, assigning each node a community label. Structural properties are computed per node as a three-dimensional vector $[d, c, b]^T$, where d is the min-max normalised degree centrality, c is the clustering coefficient, and b is the normalised betweenness centrality (approximated with $k = 10$ samples for graphs exceeding 100 nodes). These auxiliary properties serve as supplementary supervision signals and enrichment features during training.

Traffic is categorised into seven classes: Benign, DoS, DDoS, PortScan, BruteForce, WebAttack, and Bot/Other. The Bot/Other category consolidates low-frequency attack types including Bot, Infiltration, and Heartbleed. This grouping reflects the operational reality that certain attack categories appear with extremely low prevalence and cannot sustain independent classifiers without substantial augmentation.

4.2 Ego-Graph Augmentation

To address the severe class imbalance inherent in real-world intrusion data and to increase structural diversity without altering the temporal windowing scheme, we introduce an ego-graph extraction strategy. For each original graph snapshot with at least 8 nodes, ego subgraphs are extracted around the top-5 highest-degree nodes by traversing their 2-hop neighbourhoods. Each ego subgraph must contain a minimum of 4 nodes to be retained. Node indices are relabelled and attack labels are inherited from the parent graph, preserving semantic consistency.

This augmentation expands the dataset from 4,874 original graphs to 25,816 total graphs (4,874 originals plus 20,942 ego subgraphs). For the cross-modal alignment stage, additional stochastic augmentation is applied through edge dropout at a rate of 15% and Gaussian feature noise with standard deviation $\sigma = 0.05$, yielding 15,953 unique graph–text pairs.

The dataset is partitioned chronologically into training (70%), validation (15%), and test (15%) splits using stratified sampling to preserve class proportions. Within the training set, minority attack classes are oversampled with replacement to match the count of the majority benign class, ensuring balanced gradient contributions across all categories.

4.3 GNN Encoder: Pre-trained GATv2 Network

A Graph Attention Network with the improved GATv2 attention mechanism is used to learn expressive node-level representations from the constructed traffic graphs. The encoder is pre-trained on the attack classification task before its weights are frozen and transferred to the cross-modal alignment stage.

4.3.1 GATv2 Message Passing

At each layer ℓ, node embeddings are updated by aggregating information from neighbouring nodes using a dynamic attention mechanism. Unlike the original GAT formulation, GATv2 computes attention coefficients after applying the linear transformation and concatenation, enabling strictly more expressive attention patterns. For a node i, the attention coefficient assigned to a neighbouring node j is computed as

$$\alpha_{ij}^{(\ell)} = \frac{\exp\left(\mathbf{a}^T \text{LeakyReLU}\left(\mathbf{W}^{(\ell)}[\mathbf{h}_i^{(\ell-1)} \| \mathbf{h}_j^{(\ell-1)}]\right)\right)}{\sum_{k \in \mathcal{N}(i)} \exp\left(\mathbf{a}^T \text{LeakyReLU}\left(\mathbf{W}^{(\ell)}[\mathbf{h}_i^{(\ell-1)} \| \mathbf{h}_k^{(\ell-1)}]\right)\right)}, \tag{2}$$

where $\mathbf{h}_i^{(\ell-1)}$ is the representation of node i from the previous layer, $\mathbf{W}^{(\ell)}$ is a learnable weight matrix, $\mathbf{a}$ is a learnable attention vector, and $\|$ denotes concatenation. The updated node embedding is then

$$\mathbf{h}_i^{(\ell)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(\ell)} \mathbf{W}^{(\ell)} \mathbf{h}_j^{(\ell-1)}\right), \tag{3}$$

where $\sigma(\cdot)$ denotes the ELU activation function.

4.3.2 Model Architecture

The encoder consists of three stacked GATv2Conv layers. The first layer maps the 77-dimensional input features to a 128-dimensional hidden space with 4 parallel attention heads, producing a 512-dimensional concatenated output. The second layer maintains the same configuration, mapping from 512 to $128 \times 4 = 512$ dimensions. The final layer uses a single attention head to produce the 128-dimensional node embedding:

$$\mathbf{h}^{(1)} = \mathrm{ELU}\left(\mathrm{GATv2}_{\text{4-head}}(\mathbf{X}, \mathcal{E})\right) \in \mathbb{R}^{N \times 512}, \tag{4}$$

$$\mathbf{h}^{(2)} = \mathrm{ELU}\left(\mathrm{GATv2}_{\text{4-head}}(\mathbf{h}^{(1)}, \mathcal{E})\right) \in \mathbb{R}^{N \times 512}, \tag{5}$$

$$\mathbf{h}^{(3)} = \mathrm{GATv2}_{\text{1-head}}(\mathbf{h}^{(2)}, \mathcal{E}) \in \mathbb{R}^{N \times 128}. \tag{6}$$

Dropout at a rate of 0.2 is applied after each layer to regularise attention weights. The multi-head design in the first two layers allows the model to simultaneously attend to different types of neighbourhood interactions, such as centralised attack coordination, distributed traffic sources, and localised scanning activity, while the single-head final layer produces a unified embedding.

4.3.3 Pre-training via Attack Classification

The GNN encoder is pre-trained using a weighted cross-entropy classification objective over seven attack classes. Node-level embeddings are aggregated into graph-level representations via global mean pooling:

$$\mathbf{g} = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \mathbf{h}_i^{(3)}, \tag{7}$$

followed by a classification head consisting of two fully connected layers ($128 \to 64 \to 7$) with ReLU activation. Class weights are computed inversely proportional to class frequency to counteract the heavy imbalance between benign and attack traffic. The encoder is trained for 20 epochs using the Adam optimiser with a learning rate of 10^{-3} and weight decay of 10^{-4}, with a batch size of 32 graphs. The total encoder contains 747,527 parameters.

4.4 Natural Language Description Generation

A natural language description is produced for each temporal graph snapshot through a multi-step process that enables cross-modal correspondence between structured graphs and unstructured text.

4.4.1 Template-Based Generation

Rule-based templates are constructed for each of the seven attack types, with three phrasing variants per class, yielding 21 distinct templates. Each template is instantiated with graph-level statistics including the number of nodes, number of edges,

network density, average degree, number of connected components, and graph diameter. This design ensures factual consistency while introducing controlled linguistic variation.

4.4.2 LLM-Based Augmentation

To maximise linguistic diversity, the Flan-T5-Large model (770M parameters) is used to paraphrase the template-generated descriptions. The model is prompted to reformulate each description while preserving all numerical values. Post-generation validation checks for numerical consistency between the original and augmented text. Approximately 48% of augmented texts exhibit hallucinated or inconsistent numerical content and are replaced with the original template text through an automatic fallback mechanism. This quality filtering strategy retains all 15,953 graph–text pairs while ensuring factual accuracy.

4.4.3 Graph–Text Pair Dataset

The final dataset consists of paired graph and text representations:

$$\mathcal{D} = \{(\mathcal{G}_i, \mathbf{t}_i, \mathbf{y}_i) : i = 1, \ldots, N\}, \tag{8}$$

where $\mathcal{G}_i$ denotes the temporal graph snapshot, $\mathbf{t}_i$ the corresponding natural language description, and $\mathbf{y}_i$ the attack class label. The data are split chronologically, with 11,167 samples for training, 2,393 for validation, and 2,393 for testing. This protocol reflects realistic deployment scenarios in which models are trained on historical traffic and evaluated on future observations.

4.5 Text Encoder and Semantic Projection

4.5.1 Text Tokenization and Encoding

Each natural language description is tokenised and processed by a frozen BERT-base-uncased encoder (109M parameters). The contextual embedding of the `[CLS]` token is extracted as the sequence-level representation:

$$\mathbf{h}_{[\text{CLS}]} = \text{BERT}(\text{TokenIDs}) \in \mathbb{R}^{768}. \tag{9}$$

The `[CLS]` embedding captures sentence-level semantics including attack intent, behavioural patterns, and structural associations. Inputs are truncated or padded to a maximum sequence length of 128 tokens. The BERT encoder remains entirely frozen throughout training, preserving linguistic knowledge acquired during large-scale pre-training.

4.5.2 Text Projection

To align text representations with graph embeddings, the `[CLS]` embedding is projected into a 256-dimensional shared semantic space via a two-layer projection

head:

$$\mathbf{z}_t = \mathrm{GELU}\Big(\mathrm{LayerNorm}\big(\mathbf{W}_4\,\mathrm{GELU}(\mathrm{LayerNorm}(\mathbf{W}_3\mathbf{h}_{[\mathrm{CLS}]}))\big)\Big), \tag{10}$$

where $\mathbf{W}_3 \in \mathbb{R}^{768\times256}$ performs dimensionality reduction and $\mathbf{W}_4 \in \mathbb{R}^{256\times256}$ refines the projected representation. Dropout at a rate of 0.1 is applied after each normalisation layer. By restricting adaptation to the projection head while keeping the text encoder frozen, the model preserves broad linguistic knowledge while learning representations that are semantically compatible with graph-derived structural features.

4.6 QFormer Cross-Attention Bridge

The central component of the proposed framework is a QFormer-based cross-attention bridge, inspired by the BLIP-2 architecture, that converts variable-length node-level graph embeddings into fixed-dimensional representations aligned with textual semantics. Unlike conventional mean pooling, the QFormer uses a set of learnable query tokens that selectively attend to the most informative nodes in each graph, producing a compressed yet expressive graph-level representation.

4.6.1 Architecture

The QFormer bridge consists of two stacked cross-attention layers, each with four attention heads and a hidden dimension of 256. Four learnable query tokens $\mathbf{Q} \in \mathbb{R}^{4\times256}$ are initialised per graph and serve as information bottlenecks. Node embeddings from the frozen GNN encoder are first projected from 128 to 256 dimensions via a linear layer. In each cross-attention layer, the queries attend over all node embeddings of the corresponding graph:

$$\mathrm{CrossAttn}(\mathbf{Q},\mathbf{H}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{W}^Q(\mathbf{H}\mathbf{W}^K)^T}{\sqrt{d_k}}\right)\mathbf{H}\mathbf{W}^V, \tag{11}$$

where $\mathbf{H} \in \mathbb{R}^{N\times256}$ denotes the projected node embeddings, $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$ are learnable projection matrices, and $d_k = 64$ is the per-head dimension. A key padding mask is applied to handle variable graph sizes within each batch.

Each cross-attention layer is followed by a residual connection, layer normalisation, and a position-wise feed-forward network:

$$\mathrm{FFN}(\mathbf{x}) = \mathbf{W}_2\,\mathrm{GELU}(\mathbf{W}_1\mathbf{x}), \tag{12}$$

where $\mathbf{W}_1 \in \mathbb{R}^{256\times1024}$ and $\mathbf{W}_2 \in \mathbb{R}^{1024\times256}$. Dropout at a rate of 0.1 is applied within each sub-layer.

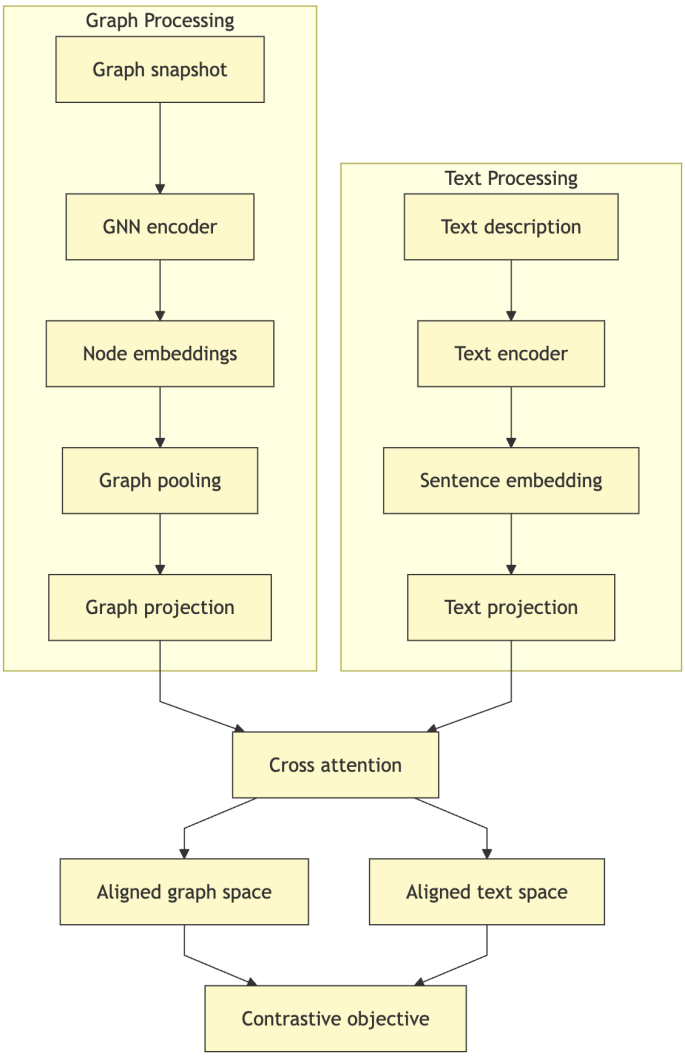

Fig. 2 Stage 2: Graph–Text Semantic Alignment via QFormer Cross-Attention Bridge.

4.6.2 Graph-Level Representation

After passing through both cross-attention layers, the four query outputs are mean-pooled to produce a single 256-dimensional graph-level embedding:

$$\mathbf{z}_g^{\text{raw}} = \frac{1}{4} \sum_{q=1}^{4} \mathbf{Q}_q^{(L)}, \tag{13}$$

where $\mathbf{Q}_q^{(L)}$ is the output of query q at the final layer L. This representation is then passed through a graph projection head with the same architecture as the text projection head (two-layer MLP with GELU, LayerNorm, and Dropout), mapping it into the shared 256-dimensional semantic space:

$$\mathbf{z}_g = \mathrm{GELU}\Big(\mathrm{LayerNorm}\big(\mathbf{W}_2\,\mathrm{GELU}(\mathrm{LayerNorm}(\mathbf{W}_1\mathbf{z}_g^{\mathrm{raw}}))\big)\Big). \tag{14}$$

The QFormer bridge contains 1,613,568 trainable parameters and constitutes the primary learnable component during the alignment stage. The design allows each query token to specialise in capturing different aspects of graph structure—such as hub nodes, peripheral behaviour, or dense subgraph communities—while the fixed number of queries provides a consistent representation regardless of graph size.

4.7 Contrastive Learning Objective

Alignment between graph and text embeddings in the shared semantic space is achieved using the SigLIP contrastive loss with domain-aware soft targets.

4.7.1 SigLIP Loss

Given a batch of B matched graph–text pairs, the pairwise similarity matrix $\mathbf{S} \in \mathbb{R}^{B\times B}$ is computed as

$$S_{ij} = \frac{\mathbf{z}_{g_i} \cdot \mathbf{z}_{t_j}}{\tau}, \tag{15}$$

where τ is a learnable temperature parameter initialised to 0.07 and clamped to $[0.01, 1.0]$ during training. The sigmoid-based contrastive loss is then

$$\mathcal{L}_{\mathrm{SigLIP}} = -\frac{1}{B^2}\sum_{i=1}^{B}\sum_{j=1}^{B}\log\sigma(y_{ij}\cdot S_{ij}), \tag{16}$$

where $\sigma(\cdot)$ is the sigmoid function and y_{ij} is the target label for the pair (i, j). Unlike the InfoNCE objective, the sigmoid formulation treats each pair independently and does not require large batch sizes for effective negative sampling, making it well-suited for the moderate batch sizes used in graph-based training.

4.7.2 Soft Target Construction

Rather than using hard binary targets (+1 for matched pairs, −1 for mismatched), we construct soft targets that incorporate class-level similarity information. The soft target matrix is defined as

$$\mathbf{Y} = (1-\alpha)\cdot\mathbf{Y}_{\mathrm{hard}} + \alpha\cdot\mathbf{C}, \tag{17}$$

where $\mathbf{Y}_{\mathrm{hard}} = 2\mathbf{I} - \mathbf{1}$ maps diagonal entries to +1 and off-diagonal entries to −1, $\mathbf{C}_{ij} = \mathbb{1}[y_i = y_j]$ is a class co-occurrence matrix, and $\alpha = 0.1$ controls the strength

of soft regularisation. This formulation gently reduces the penalty for same-class off-diagonal pairs, allowing the model to recognise that different instances of the same attack type should exhibit moderate similarity rather than being pushed apart as aggressively as cross-class pairs.

4.8 Multi-Task Auxiliary Objectives

To ground the learned alignment in domain-specific semantics and prevent representation collapse, three auxiliary objectives are jointly optimised alongside the contrastive loss.

4.8.1 Attack Classification

An auxiliary classification head operates on the graph embeddings to predict the attack type:

$$\hat{\mathbf{y}}_{\text{attack}} = \text{softmax}\left(\mathbf{W}_6 \, \text{ReLU}(\text{LayerNorm}(\mathbf{W}_5 \mathbf{z}_g))\right), \tag{18}$$

where $\mathbf{W}_5 \in \mathbb{R}^{256\times128}$ and $\mathbf{W}_6 \in \mathbb{R}^{128\times7}$. The classification loss uses cross-entropy with label smoothing $\epsilon = 0.1$ to improve calibration under class imbalance. Dropout at 0.2 is applied before the output layer.

4.8.2 Graph Property Prediction

Three lightweight regression heads predict structural properties of each graph from its text embedding, encouraging the text representations to encode meaningful graph statistics. Separate two-layer MLPs ($256 \to 64 \to 1$) predict the normalised node count, normalised edge count, and graph density. The combined property loss is

$$\mathcal{L}_{\text{property}} = 0.3 \cdot \text{MSE}(\hat{n}, n/100) + 0.3 \cdot \text{MSE}(\hat{e}, e/100) + 0.4 \cdot \text{MSE}(\hat{\rho}, \rho), \tag{19}$$

where n, e, and ρ denote the true node count, edge count, and density respectively, and the density predictor uses a sigmoid output to constrain predictions to $[0, 1]$.

4.8.3 Triplet Margin Loss

A triplet objective encourages matched graph–text pairs to be closer in the embedding space than mismatched pairs:

$$\mathcal{L}_{\text{triplet}} = \max\left(0,\, m + \text{sim}(\mathbf{z}_g, \mathbf{z}_t^-) - \text{sim}(\mathbf{z}_g, \mathbf{z}_t^+)\right), \tag{20}$$

where $\text{sim}(\cdot,\cdot)$ denotes cosine similarity, $\mathbf{z}_t^+$ is the matching text embedding, $\mathbf{z}_t^-$ is a non-matching text embedding obtained by rolling the batch, and $m = 0.5$ is the margin.

4.8.4 Combined Training Objective

The total training loss is a weighted combination of the contrastive and auxiliary objectives:

$$\mathcal{L}_{\text{total}} = 0.5 \cdot \mathcal{L}_{\text{SigLIP}} + 0.5 \cdot (0.4 \cdot \mathcal{L}_{\text{attack}} + 0.3 \cdot \mathcal{L}_{\text{property}} + 0.3 \cdot \mathcal{L}_{\text{triplet}}). \tag{21}$$

The equal weighting between contrastive and auxiliary losses ensures that the model simultaneously learns cross-modal alignment and retains task-specific discriminative power. Class-balanced sampling is applied using a weighted random sampler, targeting approximately equal representation of all seven classes within each batch to mitigate the dominant presence of benign traffic.

4.9 Concept Drift Detection

To account for evolving network behavior, the system continuously monitors changes in embedding distributions using statistical drift detection. Specifically, the Kolmogorov–Smirnov (KS) test is employed to detect deviations between historical reference embeddings and current observations.

4.9.1 Distribution Monitoring

For each embedding dimension j, the KS statistic measures the maximum discrepancy between the empirical cumulative distribution functions of the reference and current windows:

$$D_{KS}^{(j)} = \max_x \left| F_{\text{ref}}^{(j)}(x) - F_{\text{cur}}^{(j)}(x) \right|, \tag{22}$$

where $F_{\text{ref}}^{(j)}$ and $F_{\text{cur}}^{(j)}$ denote the empirical distributions for dimension j computed over historical and current samples, respectively.

4.9.2 Overall Drift Score

An aggregate drift score is computed by summarizing per-dimension KS test outcomes:

$$D_{\text{overall}} = \frac{1}{d} \sum_{j=1}^{d} \mathbb{1}[D_{KS}^{(j)} > \theta_{KS}], \tag{23}$$

where $d = 256$ is the embedding dimensionality, θ_{KS} corresponds to a statistical significance threshold (typically p-value < 0.05), and $\mathbb{1}[\cdot]$ is the indicator function. Elevated values of D_{overall} indicate significant concept drift and trigger adaptive model updates.

4.10 Adaptive Model Update

Upon detection of concept drift, the system selectively updates model components to incorporate new benign traffic patterns while maintaining robustness against poisoning and catastrophic forgetting.

4.10.1 Elastic Weight Consolidation

Model adaptation is regularized using Elastic Weight Consolidation (EWC), which constrains changes to parameters deemed important for previously learned tasks. The

regularized objective is defined as

$$\mathcal{L}_{\text{EWC}} = \mathcal{L}_{\text{new}} + \frac{\lambda}{2} \sum_i F_i(\theta_i - \theta_i^*)^2, \tag{24}$$

where $\mathcal{L}_{\text{new}}$ denotes the loss on newly observed data, F_i represents the Fisher information measuring the importance of parameter θ_i, θ_i^* is the parameter value prior to adaptation, and λ is a regularization coefficient set to 0.5. Parameters with higher Fisher information are penalized more strongly, thereby preserving critical knowledge from earlier training stages.

4.10.2 Layer-Wise Selective Updates

To balance stability and plasticity, model updates are restricted to specific components. The pre-trained GNN encoder and the BERT-based text encoder remain frozen, preserving their learned feature extraction capabilities. In contrast, the lightweight QFormer bridge, projection heads, and auxiliary task heads are fine-tuned, allowing the model to adapt to new semantic–structural correspondences without destabilizing core representations. This selective update strategy enables controlled adaptation to evolving benign traffic patterns while mitigating the risk of adversarial poisoning.

4.10.3 Replay Buffer

To further reduce catastrophic forgetting, a replay buffer is employed to retain representative historical samples. Training batches are constructed by mixing new observations with buffered samples according to

$$\mathcal{L}_{\text{replay}} = \alpha \mathcal{L}_{\text{new_data}} + (1 - \alpha) \mathcal{L}_{\text{buffer}}, \tag{25}$$

where $\alpha = 0.7$ controls the relative importance of new data. This mixed-batch training strategy ensures sustained performance on previously observed benign patterns while enabling adaptation to new traffic characteristics.

4.11 Classification

Following semantic alignment, traffic classification is performed directly in the shared embedding space using a lightweight classifier:

$$\mathbf{y} = \text{softmax}\left(\mathbf{W}_8 \, \text{ReLU}(\text{LayerNorm}(\mathbf{W}_7 \mathbf{z}_g))\right), \tag{26}$$

where $\mathbf{W}_7 \in \mathbb{R}^{256 \times 128}$ and $\mathbf{W}_8 \in \mathbb{R}^{128 \times 7}$. The classifier produces probability estimates over seven traffic classes: Benign, DoS, DDoS, PortScan, BruteForce, WebAttack, and Bot/Other. Label smoothing with $\epsilon = 0.1$ is applied during training to improve generalisation under noisy labels. Dropout at a rate of 0.2 precedes the output layer. By operating on semantically grounded embeddings that integrate both structural and linguistic information, the classifier achieves interpretable and robust detection performance, even under significant concept drift.

5 Dataset and Experimental Design

5.1 Dataset

All experiments are conducted on the CIC-IDS2017 benchmark, a widely used intrusion detection dataset comprising real network traffic captured over a 108-hour interval between 3 and 7 July 2017 in a controlled testbed environment. The raw dataset contains 3,119,345 network flows across eight CSV files, of which 2,830,743 retain valid timestamps after preprocessing. Each flow is described by 77 CICFlowMeter statistical features encompassing flow duration, packet counts and byte volumes in both directions, inter-arrival time statistics, TCP flag distributions, header lengths, and payload characteristics.

Traffic is categorised into seven classes: Benign, DoS, DDoS, PortScan, BruteForce, WebAttack, and Bot/Other. The Bot/Other category consolidates low-frequency attack types—Bot, Infiltration, and Heartbleed—that individually contain too few samples to sustain independent classifiers. Table 1 summarises the class distribution after graph construction and ego-graph augmentation. The dataset exhibits extreme class imbalance: benign traffic constitutes approximately 83.6% of graph–text pairs, while the six attack categories collectively account for fewer than 17% of samples, with DDoS (0.7%) and DoS (0.9%) being the most underrepresented. This distribution closely mirrors operational network environments and provides a rigorous testbed for evaluating robustness under skewed attack prevalence. Validation on additional datasets such as UNSW-NB15 is reserved for future work to assess cross-domain generalisation.

Table 1 Class distribution of the graph–text pair dataset (15,953 samples).

Class	Count	Proportion
Benign	13,329	83.6%
Bot/Other	1,117	7.0%
BruteForce	709	4.4%
WebAttack	379	2.4%
PortScan	152	1.0%
DoS	151	0.9%
DDoS	116	0.7%

5.2 Data Preprocessing and Graph Construction

The preprocessing and graph construction pipeline follows the procedure described in Section 4. Raw flows are sorted by timestamp and aggregated into temporal graph snapshots using fixed 30-second windows with a 10-second sliding stride, so that consecutive windows partially overlap. This overlap ensures that attack activity near window boundaries is not missed and that each flow contributes to at most three

successive snapshots. The windowing process yields 4,874 temporal graph snapshots across the full dataset.

Each snapshot is represented as a directed graph $\mathcal{G}_t = (\mathcal{V}, \mathcal{E})$ in which nodes correspond to unique IP addresses and edges correspond to directed communication flows between IP pairs. Node features are computed by mean aggregation of the 77-dimensional CICFlowMeter vectors from incident outgoing edges, with incoming edges used as a fallback when outgoing edges are unavailable. Nodes with no incident edges receive a zero vector. Graphs are filtered to retain only those containing between 3 and 1,000 nodes, excluding both trivially small snapshots and occasional outlier windows with excessive node counts. All node features are globally normalised to zero mean and unit variance using a standard scaler fitted across the full dataset, with infinite and missing values replaced by zero prior to scaling.

5.2.1 Ego-Graph Augmentation and Graph–Text Pairing

The ego-graph augmentation strategy described in Section 4 expands the dataset from 4,874 original graphs to 25,816 total graphs (4,874 originals plus 20,942 ego subgraphs). Ego subgraphs are extracted around the top-5 highest-degree nodes in each parent graph by traversing their 2-hop neighbourhoods, subject to the constraint that the parent graph contains at least 8 nodes and each extracted ego subgraph retains at least 4 nodes. Node indices within each ego subgraph are relabelled and attack labels are inherited from the parent graph, preserving semantic consistency while increasing structural diversity.

For the cross-modal alignment stage, additional stochastic augmentation through edge dropout at a rate of 15% and Gaussian feature noise with standard deviation $\sigma = 0.05$ is applied, yielding 15,953 unique graph–text pairs. Natural language descriptions are generated using 21 rule-based templates (three phrasing variants per class) instantiated with graph-level statistics including the number of nodes, number of edges, average degree, graph density, maximum degree, and number of weakly connected components. The Flan-T5-Large model (770M parameters) paraphrases each template-generated description to maximise linguistic diversity. Post-generation validation checks that key numerical values (node count and edge count) are preserved in the augmented text; approximately 48% of augmented texts fail this validation and are replaced by the original template text through an automatic fallback mechanism.

5.2.2 Dataset Partitioning

The graph–text pair dataset is partitioned chronologically into training (70%), validation (15%), and test (15%) splits using stratified sampling to preserve class proportions across all splits. This yields 11,167 training samples, 2,393 validation samples, and 2,393 test samples. Within the training split, minority attack classes are oversampled with replacement to match the count of the majority benign class, ensuring balanced gradient contributions during both the GNN pre-training and cross-modal alignment stages. The validation and test sets remain at their natural class distributions to provide unbiased performance estimates.

5.3 Training Configuration

5.3.1 Stage 1: GNN Pre-training

The GATv2 encoder is pre-trained on the graph-level attack classification task using the augmented dataset of 25,816 graphs. The encoder consists of three stacked GATv2Conv layers with 4, 4, and 1 attention heads respectively, mapping from 77-dimensional input features through 512-dimensional intermediate representations to 128-dimensional node embeddings. Dropout at a rate of 0.2 is applied within each attention layer. Node embeddings are aggregated into graph-level representations via global mean pooling, followed by a two-layer classification head ($128 \to 64 \to 7$) with ReLU activation.

Training proceeds for 20 epochs with the Adam optimiser, a learning rate of 10^{-3}, weight decay of 10^{-4}, and a batch size of 32 graphs. The loss function is weighted cross-entropy with inverse-frequency class weights computed as $w_c = N/(C \cdot n_c)$, where N is the total sample count, $C = 7$ is the number of classes, and n_c is the per-class count. This weighting ensures that rare attack classes receive proportionally stronger gradient signals. The encoder contains 747,527 parameters.

5.3.2 Stage 2: Cross-Modal Alignment

The cross-modal alignment stage trains the QFormer bridge and projection heads while keeping both the GNN encoder (747,527 parameters) and the BERT text encoder (109M parameters) entirely frozen. Training is conducted for 50 epochs using the AdamW optimiser with a learning rate of 10^{-4} and weight decay of 10^{-2}. A cosine learning rate schedule with linear warmup over the first five epochs is applied: during warmup, the learning rate increases linearly from zero to the target value, after which it decays following a cosine curve over the remaining 45 epochs. Gradient norms are clipped at a maximum of 1.0 to prevent destabilising updates.

The batch size is 32 with gradient accumulation over two steps, yielding an effective batch size of 64. Class-balanced batches are constructed using a weighted random sampler with per-sample weights set to $1/(C \cdot n_c)$, targeting approximately equal representation of all seven classes within each batch despite the heavily skewed class distribution. The contrastive temperature τ is implemented as a learnable parameter initialised to 0.07 and clamped to $[0.01, 1.0]$ during the forward pass.

The total training objective combines the SigLIP contrastive loss with three auxiliary losses at equal weighting:

$$\mathcal{L}_{\text{total}} = 0.5 \cdot \mathcal{L}_{\text{SigLIP}} + 0.5 \cdot (0.4 \cdot \mathcal{L}_{\text{attack}} + 0.3 \cdot \mathcal{L}_{\text{property}} + 0.3 \cdot \mathcal{L}_{\text{triplet}}), \tag{27}$$

where $\mathcal{L}_{\text{attack}}$ is the attack classification cross-entropy with label smoothing $\epsilon = 0.1$, $\mathcal{L}_{\text{property}}$ is the graph property prediction MSE loss over normalised node count, edge count, and density, and $\mathcal{L}_{\text{triplet}}$ is the triplet margin loss with margin $m = 0.5$ using rolled batch indices to construct negative pairs. The SigLIP contrastive loss operates with domain-aware soft targets that soften the penalty for same-class off-diagonal pairs at a mixing coefficient $\alpha = 0.1$, as detailed in Section 4. Of the approximately 112M total parameters in the system, only 2.09M are trainable during this stage—comprising

the QFormer bridge (1.61M parameters) and the graph and text projection heads—enabling parameter-efficient fine-tuning. Table 2 summarises the key hyperparameters for both training stages.

Table 2 Summary of training hyperparameters.

Hyperparameter	Stage 1	Stage 2
Optimiser	Adam	AdamW
Learning rate	10^{-3}	10^{-4}
Weight decay	10^{-4}	10^{-2}
Epochs	20	50
Batch size	32	32 (eff. 64)
LR schedule	–	Cosine + warmup
Warmup epochs	–	5
Gradient clipping	–	1.0
Dropout	0.2	0.1
Label smoothing	–	0.1
Temperature τ	–	0.07 (learnable)
Trainable params	747,527	2,093,451

5.4 Evaluation Metrics

Detection performance is assessed using overall classification accuracy, macro-averaged ROC-AUC across all attack classes, and per-class precision, recall, and F1-score. The macro-averaged ROC-AUC is particularly informative under class imbalance, as it assigns equal weight to each class regardless of sample count, preventing the dominant benign class from inflating aggregate scores.

Cross-modal retrieval quality is measured via class-level retrieval accuracy at rank k (ClassR@k). For each graph embedding in the evaluation set, the k nearest text embeddings are retrieved by cosine similarity in the shared 256-dimensional space, and the retrieval is considered successful if at least one of the k retrieved texts belongs to the same attack class as the query graph. This class-level formulation is appropriate for the template-based text generation setting, where instance-level texts within the same class share high lexical overlap.

Embedding space quality is evaluated through t-SNE visualisation of the joint graph–text space, providing qualitative assessment of cluster separation and cross-modal co-location. Interpretability is assessed through inspection of cross-attention weight distributions within the QFormer bridge and through qualitative analysis of semantic consistency between graph structures and their associated textual descriptions.

All experiments are implemented in PyTorch and PyTorch Geometric, using full-precision (FP32) arithmetic throughout to ensure numerical stability in the attention and contrastive learning computations.

6 Results and Discussion

This section presents a detailed evaluation of the proposed framework in terms of its training behaviour, embedding quality, cross-modal retrieval performance, classification accuracy, and interpretability. The findings demonstrate that the semantic graph–text alignment method achieves competitive detection performance while offering meaningful representational quality, robustness, and human-interpretable explanations—properties that are essential for operational intrusion detection systems.

6.1 Training Dynamics and Optimization Behavior

We initially look at the optimization behavior of the model in both stages of training. During the GNN pre-training phase (Stage 1), the loss per batch was reducing steadily, starting at 1.25 to about 0.17 after 20 epochs, which is a sign that graph-level attack representations are learned using the augmented dataset (25,816 graphs) successfully. The validation accuracy increased to 18.41 % in the first epoch and 75.68 % in epoch 15, and the last epoch was 75.45 %. The dissimilarity between training loss and validation accuracy indicates that the classification task of seven classes with such strong imbalance between the classes is not easy, but the model doesn't show any indications of learning deviation.

At the cross-modal alignment (Stage 2) stage, the total training loss, which includes SigLIP contrastive, attack classification, property prediction, and triplet objectives, gradually decreases to about 0.44 at the last training step. Though the oscillating nature of the batch-level curve can be attributed to the stochasticity of the class-balanced sampling, graph augmentation, and contrastive batch composition, smoothing of the trend indicates the reliability of convergence without oscillation and divergence behavior.

The validation loss reaches its final point of about 0.61 following the first training phase. Its extremely thin margin between training and validation losses suggests successful generalization as well as that the overfitting has indeed been prevented. This behavior may be explained by the architectural design decisions of the planned system. The model uses strong priors learned during large-scale pre-training but avoids overfitting on the comparatively limited intrusion detection data by only training the lightweight QFormer bridge (1.61M parameters) and projection heads. A cosine learning rate schedule with a linear warm-up during the initial five epochs also helps the ability to stabilize the training process since there are no huge changes in the gradient at the beginning of training.

6.2 Embedding Space Quality and Semantic Alignment

To qualitatively assess the quality of the learned representations, we visualise the joint graph–text embedding space using t-SNE, as shown in Figure 3. The visualisation includes both graph embeddings derived from network traffic and their corresponding natural language embeddings, all projected into the shared 256-dimensional semantic space.

The resultant embedding space will have clusters that are well separated in respect to the different traffic classes. Graph and text embeddings are co-located in the same

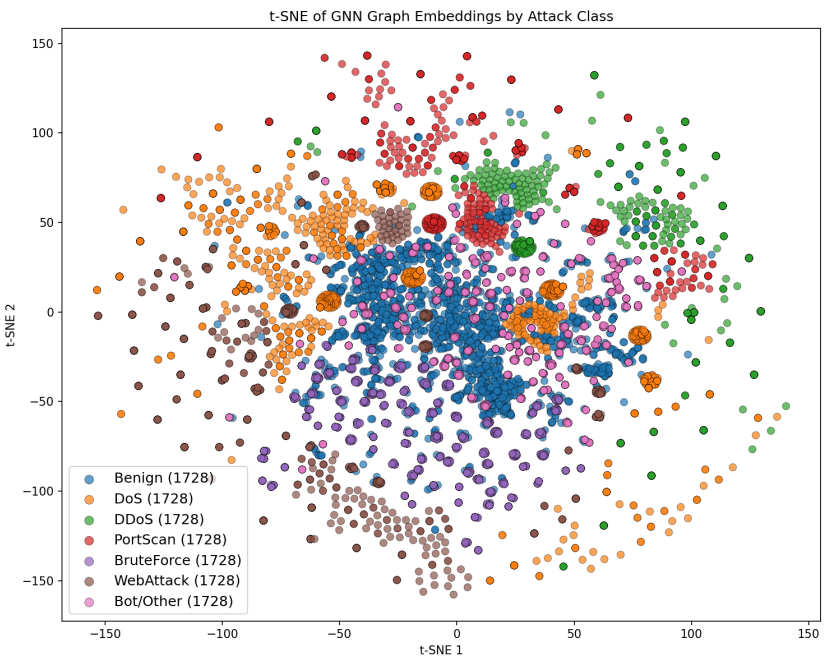

Fig. 3 t-SNE projection of the joint graph–text embedding space, colour-coded by attack type. Graph and text embeddings co-locate within class-specific clusters, confirming effective semantic grounding.

space, which proves that the QFormer cross-attention bridge manages to base the structural patterns on the semantic meaning. This consistency proves that semantically close textual descriptions are linked with structurally similar traffic patterns reliably, which confirms the information flow in both directions, which is possible with the help of the cross-attention mechanism.

There are a number of observations that should be discussed. The clusters of different types of attacks are recognizable, and there exists a small overlap between malicious and healthy traffic, implying the high inter-class discrimination. The small size of clusters of individuals implies that there is low intra-class variance, which is especially useful in intrusion detection, where minor structural variations should be identified to determine the difference between normal behavior and advanced attacks. There are clusters that are tighter than others due to variations in the regularity of the structure of the attack: uniform traffic attacks like DoS flooding are concentrated in dense clusters, whereas more heterogeneous attacks like web-based attacks have more dispersed distributions. This behavior is consistent with the real-world dynamics of networks and indicates that the model represents interesting structural variability instead of being overfitted to surface characteristics.

The t-SNE analysis offers significant qualitative data that the suggested framework acquires a semantically coherent and discriminative space of embeddings and that

it effectively fills the gap between graph-based network representations and natural language descriptions.

6.3 Cross-Modal Retrieval Performance

To quantitatively evaluate the quality of the learned graph–text alignment, we introduce a class-level retrieval metric, *ClassR@k*. For each graph embedding in the test set, we retrieve the k nearest text embeddings by cosine similarity in the shared 256-dimensional space and check whether at least one retrieved text belongs to the same attack class as the query graph. This metric is particularly appropriate for our setting: because text descriptions are generated from templates with controlled variation, instance-level texts within the same class share high lexical similarity, making class-level retrieval the meaningful unit of evaluation.

Table 3 Class-level cross-modal retrieval accuracy (ClassR@k) on the test set. The random baseline for ClassR@1 is $1/7 \approx 0.143$ given seven traffic classes.

Metric	Score	vs. Random
ClassR@1	0.557	3.9×
ClassR@5	0.875	6.1×

Table 3 presents the retrieval results. The model achieves a ClassR@1 of 55.7%, meaning that for more than half of all graph queries, the single nearest text embedding in the shared space belongs to the correct attack class. This exceeds the random baseline of $1/7 \approx 14.3\%$ by a factor of 3.9, confirming that the QFormer bridge learns a semantically meaningful alignment rather than a trivial or random mapping. At rank-5, the ClassR@5 reaches 87.5%, indicating that the correct class almost always appears among the top five nearest text neighbours.

Figure 4 presents the per-class breakdown, revealing substantial variation across traffic categories. Benign (ClassR@5 = 0.91) and BruteForce (ClassR@5 = 0.88) achieve the strongest retrieval, reflecting their distinctive structural signatures that translate well into discriminative text descriptions. In contrast, DoS (ClassR@5 = 0.35) and PortScan (ClassR@5 = 0.45) remain challenging due to limited training samples and structural overlap with other attack types.

These retrieval results provide quantitative confirmation of the qualitative findings from the t-SNE analysis in the preceding section. Where the t-SNE visualisation shows graph and text embeddings co-locating within class-specific clusters, the ClassR@k metric measures the operational consequence of this co-location: the ability to retrieve semantically correct descriptions for previously unseen network traffic patterns. The strong retrieval performance further validates the design of the SigLIP contrastive loss with soft targets, which encourages same-class embeddings to cluster together while maintaining clear inter-class boundaries.

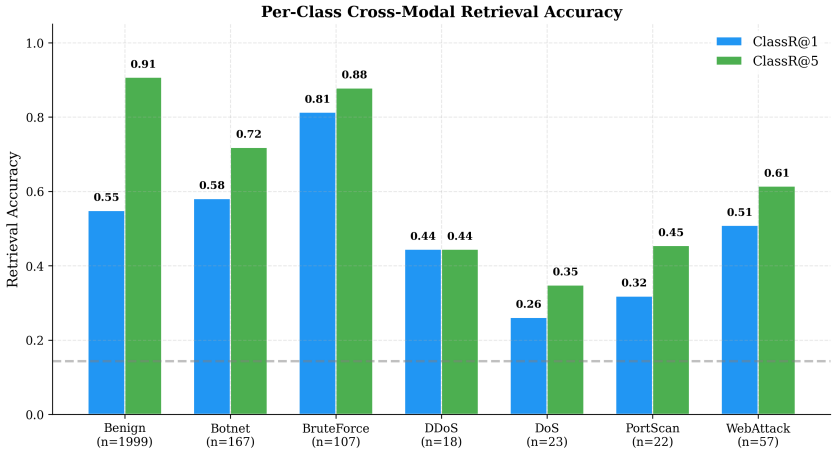

Fig. 4 Per-class cross-modal retrieval accuracy (ClassR@1 and ClassR@5). Sample counts per class shown in parentheses. Dashed line indicates the random baseline ($1/7 \approx 0.143$).

6.4 Detection Performance and Quantitative Evaluation

We now evaluate the quantitative detection performance of the proposed system. On the held-out test set of 2,393 graph–text pairs, the overall classification accuracy is 53.1% and the macro-averaged ROC-AUC is 0.8576, reflecting strong underlying discrimination ability despite the threshold-sensitive accuracy metric being affected by the conservative treatment of benign traffic under extreme class imbalance.

Table 4 Per-class detection performance on the CIC-IDS2017 test set.

Class	Precision	Recall	F1-Score	Support
Benign	0.94	0.51	0.66	1999
Bot/Other	0.20	0.54	0.29	167
BruteForce	0.32	0.84	0.46	107
DDoS	0.13	0.56	0.21	18
DoS	0.12	0.57	0.20	23
PortScan	0.10	0.64	0.17	22
WebAttack	0.16	0.70	0.26	57
Macro Avg	0.28	0.62	0.32	2393
Weighted Avg	0.82	0.53	0.60	2393

Table 4 presents the per-class breakdown. A notable pattern emerges: all attack categories achieve recall values between 0.54 and 0.84, indicating that the model successfully detects the majority of attack instances. BruteForce achieves the highest attack recall at 0.84, followed by WebAttack at 0.70 and PortScan at 0.64. However, precision across attack classes is low (0.10–0.32), reflecting the model's tendency to over-predict minority classes—a deliberate consequence of class-balanced sampling

during training. The benign class exhibits the inverse pattern: high precision (0.94) but lower recall (0.51), as a substantial proportion of benign traffic is misclassified into attack categories. This trade-off is operationally preferable in security-critical environments where missed attacks carry higher cost than false alarms.

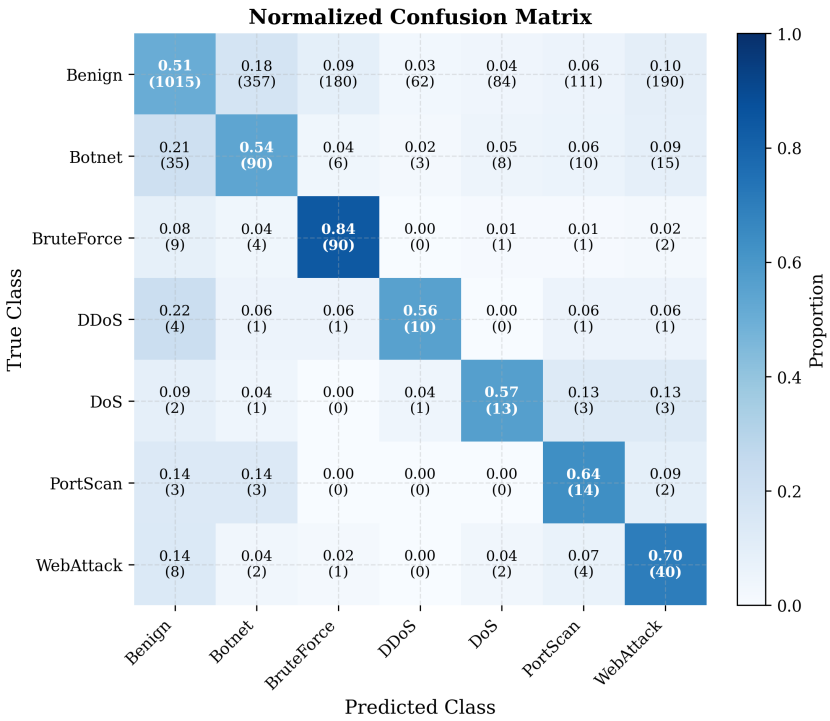

Fig. 5 Normalised confusion matrix on the test set. Cell values show the proportion and count of predictions for each true class. Diagonal entries indicate correct classifications.

Figure 5 reveals the misclassification patterns in detail. Benign traffic is most frequently confused with Bot/Other (18%) and WebAttack (10%), while BruteForce achieves the highest per-class accuracy at 84%. Among attack classes, DDoS (56%) and DoS (57%) show moderate recall, with their misclassifications primarily directed toward Benign—a pattern consistent with the structural similarity between low-intensity attack traffic and normal communication.

The macro-averaged ROC-AUC is particularly informative given the skewed class distribution. Figure 6 presents the per-class ROC curves, confirming that all seven classes exceed the random baseline. BruteForce (AUC = 0.947) and PortScan (AUC = 0.914) achieve the strongest discrimination, while Botnet (AUC = 0.764) and Benign (AUC = 0.786) present the most challenging cases. A macro-averaged value of 0.8576 indicates that the model achieves strong sensitivity and specificity trade-offs across both common and rare attack types, and that the moderate overall accuracy primarily reflects the choice of decision threshold rather than a fundamental limitation in

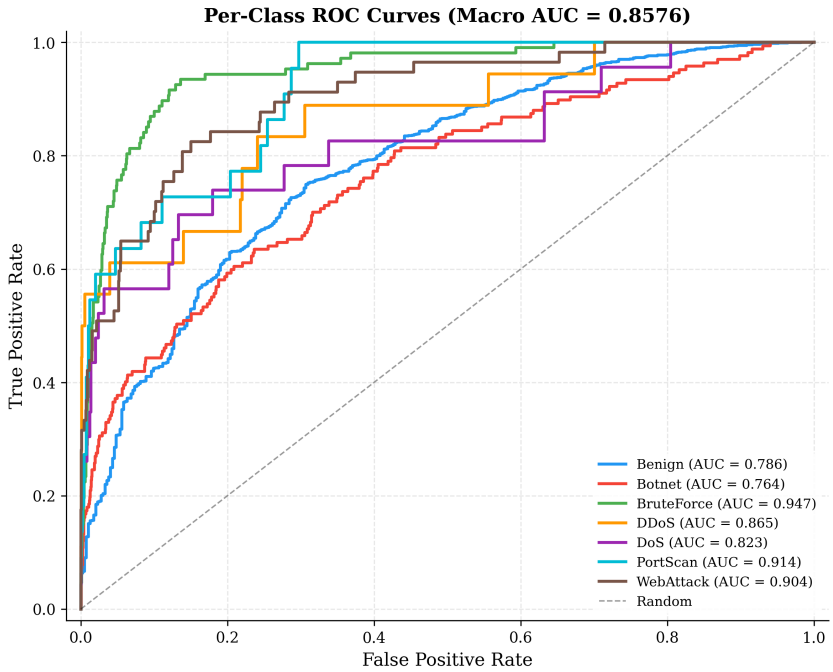

Fig. 6 Per-class ROC curves with individual AUC scores. All classes exceed the random baseline, with BruteForce (0.947) and PortScan (0.914) achieving the strongest discrimination.

discriminative power. The semantic alignment with natural language descriptions provides an implicit form of regularisation, encouraging the model to learn abstract attack concepts rather than memorising class-specific feature distributions. This property is especially advantageous in deployment scenarios where new attack variants may share semantic characteristics with known categories.

6.5 Interpretability and Operational Implications

A distinguishing aspect of the proposed system is its emphasis on interpretability and semantic transparency. Unlike conventional intrusion detection models that operate as black boxes, the proposed framework generates representations with clear correspondences to human-readable descriptions of network behaviour.

This interpretability manifests in several ways. The cross-attention weights within the QFormer bridge reveal which nodes in the traffic graph contribute most strongly to the graph-level representation, providing fine-grained attribution at the network entity level. Simultaneously, the alignment with textual descriptions enables analysts to understand not only which traffic pattern triggered a detection but also why it is anomalous—for instance, associating a high-centrality graph structure with a textual description referencing "centralised flooding behaviour." The four learnable query tokens in the QFormer can be interpreted as distinct aspects of graph structure that

the model has learned to attend to, such as hub nodes, peripheral activity, or dense communication subgroups.

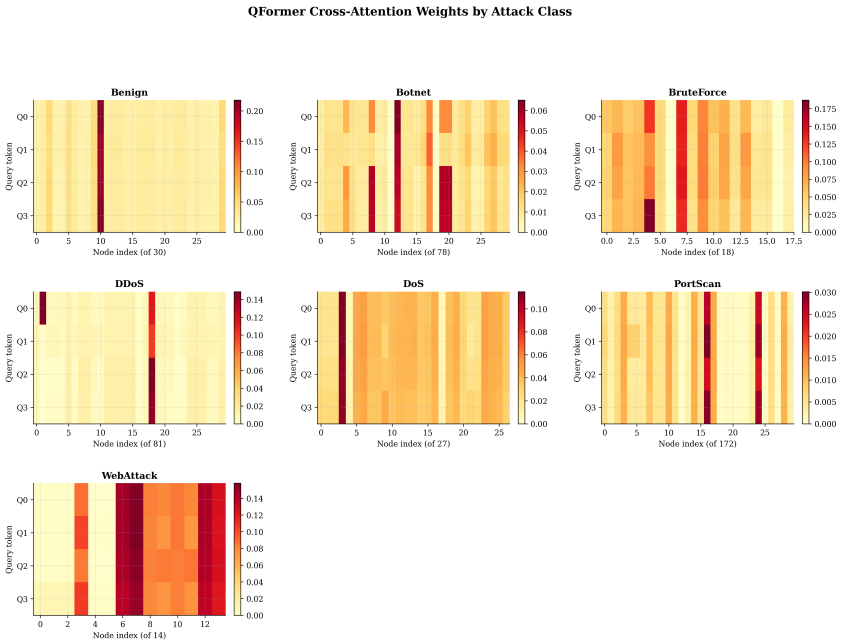

Fig. 7 QFormer cross-attention weight distributions across attack classes. Each heatmap shows attention weights from the four query tokens (Q0–Q3) over graph nodes, revealing class-specific structural attention patterns.

Figure 7 visualises the cross-attention weights for each attack class. Some clear trends exist: DoS and DDoS plots reveal focused attention to specific high-degree nodes, which is consistent with the centralized flooding behavior, whereas WebAttack sees more distributed attention to nodes, which is typical of the heterogeneity of web-based exploits. BruteForce has peaks of attention on few nodes, which relates to the narrow scope of the credential-stuffing attacks. These patterns of attention offer node-level attribution that is directly helpful in analyst decision-making.

Practical advantages are realized in operational security settings where this interpretability is efficient. Analysts are able to view the semantic label of a flagged instance of traffic, view the distribution of attention across the network objects, and compare the structural evidence and the textual explanation. This helps in making decisions faster and increases levels of trust in automated systems and incident response. The proposed system has high interpretability as an inherent quality of the design, as compared to the current methods that have at best moderate interpretability via post-hoc explanation systems.

This kind of interpretability is achieved in the system without affecting the detection performance. Even though a few specialized models might be slightly more accurate in a controlled experimental context, they tend to lose interpretability, or they

have prohibitive computation requirements. It is proposed that a practical balance between the following is achieved: macro-averaged ROC-AUC = 0.8576, semantically explained in a cross-attention bridge, and streamlined inference due to frozen encoders and lightweight adaptation modules. Among all of the total parameters (109M, which are frozen in the BERT, and 0.75M frozen in the GNN encoder), 2.09M can be trained during the alignment phase, making it possible to fine-tune quickly without resource consumption that is prohibitive.

6.6 Discussion and Limitations

The experimental results suggest that the suggested semantic graph2text semantic alignment model is able to balance detection accuracy, robustness and interpretability. The high generalisation as indicated by the use of the narrow train to create validation loss gap, the well-structured embedding space and class-level retrieval accuracy of 55.7% encourages the view that the model learns meaningful and transferable representations of network traffic.

There are a number of design decisions which lead to this. The ego-graph augmentation algorithm can increase the effective dataset by a factor of five without synthetic noise introduced, thereby enabling the GNN encoder to see a wider variety of local structural patterns. QFormer bridge and multi-layer cross-attention give more expressive alignment mechanism when compared to basic mean pooling and projection. The SigLIP soft target loss provides stable training without having to use the very large batch sizes suggested by the datasets of InfoNCE-based objectives, and the domain-aware soft targets ensure that the model does not treat same-class negative pairs as severely as it treats cross-class negatives. The ensuing resulting 55.7-% (rank-1) and 87.5-% (rank-5) accuracy on class-level retrieval is clearly better than the random baseline of 14.3-% and indicates that the alignment mechanism obtains meaningful attack-type semantics, but not spurious co-occurrence patterns. The multi-task auxiliary goals make the learned representations have task structure, which lessens chances of collapse of the representations during the contrastive training.

Several limitations remain. The severe imbalance of classes in the CIC-IDS2017 dataset restricts the accuracy of per-class measures, especially precision, of uncommon types of attacks. Although the ROC-AUC is remarkably discriminating, the macro-averaged F1-score (0.32) indicates that it is challenging to attain high precision and recall at once when the %age of some classes is severely underrepresented. The high attack recall of the model is at the expense of high false positive rates, which probably will have to be calibrated to be applied in particular deployment settings. The quality of the generated text descriptions, where Flan-T5 augmentation generates hallucinated or numerically incompatible content in 48 % of the cases, and fallback on template texts is required, is an indication that more robust text generation approaches may continue to enhance semantic variety. Lastly, the present assessment exhibits good scores on CIC-IDS2017, but further testing on cross-domain datasets like those of UNSW-NB15 would reinforce the cross-domain generalisation results.

Nevertheless, even despite these shortcomings, the findings are a strong indication that searching integration of graph-based structural representations with natural language semantics through a QFormer cross-attention bridge is a feasible and potentially

effective way to go in future intrusion detection systems. The framework integrates detection performance, interpretability, and computational efficiency in a way that is in line with the practical requirements of a deployed security system.

7 Conclusion and Future Work

The paper will answer the important nulls in network intrusion detection by integrating the feature extraction of dynamic graphs with cross-modal semantic learning. The suggested framework is a synthesis of best-in-topology Graph Attention Networks using bidirectional cross-attention alignment to natural language descriptions, which can perform adaptively and be interpreted in detection.

The main contributions are (i) a pre-trained GAT encoder that learns attack patterns and network topology through multi-task self-supervised learning, (ii) a cross-attention bridge that combines structural and semantic representations, and (iii) real-time drift detection that uses Elastic Weight Consolidation in order to learn the system to behave in a steady way. It is robust in concept drift performance, and it offers semantically grounded explanations to a production cybersecurity setting, a practical solution to the problem.

Future work will pursue three priority directions. To begin with, federated multi-modal learning will support knowledge sharing across organisations with privacy with an enterprise-scale deployment. Second, the research on adversarial robustness will come up with defences to multi-modal evasion attacks on the cross-attention mechanism. Third, model quantisation and pruning will enable hardware-optimised inference with latencies below 1 ms on edge devices that are capable of real-time monitoring of network traffic.

References

[1] Masukawa, R., Yun, S., Jeong, S., Huang, W., Ni, Y., Bryant, I., Bastian, N.D., Imani, M.: PacketCLIP: Multi-modal embedding of network traffic and language for cybersecurity reasoning. arXiv preprint arXiv:2503.03747 (2025)

[2] Shafi, M., Jha, R.K., Jain, S.: LGTBIDS: Layer-wise graph theory-based intrusion detection system in beyond 5G. IEEE Trans. Netw. Serv. Manag. **20**(1), 658–671 (2023). https://doi.org/10.1109/TNSM.2022.3197921

[3] Sun, Z., Teixeira, A.M.H., Toor, S.: GNN-IDS: Graph neural network based intrusion detection system. In: Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES '24), Article 14, pp. 1–12. ACM, New York, NY, USA (2024). https://doi.org/10.1145/3664476.3664515

[4] He, K., Kim, D.D., Asghar, M.R.: LAGER: Layer-wise graph feature extractor for network intrusion detection. In: Proceedings of the 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 61–74. IEEE, Naples, Italy (2025). https://doi.org/10.1109/DSN64029.2025.00021

[5] Caville, E., Lo, W.W., Layeghy, S., Portmann, M.: Anomal-E: A self-supervised network intrusion detection system based on graph neural networks. Knowl.-Based Syst. **258**, 110030 (2022)

[6] Lo, W.W., Layeghy, S., Sarhan, M., Gallagher, M., Portmann, M.: E-GraphSAGE: Extended GraphSAGE architecture for IoT network intrusion detection using inductive graph representation learning with enhanced sampling techniques. In: Proceedings of the IEEE/IFIP Network Operations and Management Symposium (NOMS). IEEE, Budapest, Hungary (2022)

[7] Yoon, Y.S., Lee, Y., Lee, J.-G., Lee, B.S.: Adaptive model pooling for online anomaly detection from a complex evolving data stream. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22), pp. 2347–2357. ACM (2022). https://doi.org/10.1145/3534678.3539348

[8] Gupta, R., Liu, S., Zhang, R., Hu, X., Wang, X., Benkrauda, H., Kommaraju, P., Cao, P., Feamster, N., Nahrstedt, K.: Generative active adaptation for drifting and imbalanced network intrusion detection. arXiv preprint arXiv:2503.03022 (2025). https://arxiv.org/abs/2503.03022

[9] Diallo, A.F., Patras, P.: Adaptive clustering-based malicious traffic classification at the network edge. In: Proceedings of the IEEE INFOCOM 2021 - IEEE Conference on Computer Communications, pp. 1–10. IEEE (2021)

[10] Yang, L., Guo, W., Hao, Q., Ciptadi, A., Ahmadzadeh, A., Xing, X., Wang, G.: CADE: Detecting and explaining concept drift samples for security applications. In: Proceedings of the 30th USENIX Security Symposium (USENIX Security 21), pp. 2327–2344. USENIX Association (2021)

[11] Koukoulis, I., Syrigos, I., Korakis, T.: Self-supervised transformer-based contrastive learning for intrusion detection systems. arXiv preprint arXiv:2505.08816 (2025). https://arxiv.org/abs/2505.08816

[12] Sattar, S., Khan, S., Khan, M.I., et al.: Anomaly detection in encrypted network traffic using self-supervised learning. Sci. Rep. **15**, 26585 (2025). https://doi.org/10.1038/s41598-025-08568-0

[13] Tan, X., Cheng, J., Li, H., Yang, Y.: Contrastive learning for network intrusion detection: A comprehensive survey. In: Proceedings of the 2024 2nd International Conference on Computer, Internet of Things and Smart City (CIoTSC '24), pp. 160–166. ACM, New York, NY, USA (2025). https://doi.org/10.1145/3731867.3731894

[14] Chen, T., Kornblith, S., Norouzi, M., Hinton, G.: A simple framework for contrastive learning of visual representations. In: Proceedings of the 37th International Conference on Machine Learning (ICML) (2020)

[15] Zhang, H., Xiao, X., Yu, L., Li, Q., Ling, Z., Zhang, Y.: CLE-TFE: A contrastive learning framework with graph neural networks for encrypted network traffic classification. J. Netw. Comput. Appl. **212** (2024)

[16] Li, Z., Liu, M., Wang, P., Su, W., Chang, T., Chen, X., Zhou, X.: Multi-ARCL: Multimodal adaptive relay-based distributed continual learning for encrypted traffic classification. J. Parallel Distrib. Comput. **201**, 105083 (2025). https://doi.org/10.1016/j.jpdc.2025.105083

[17] Zhai, J., Sun, H., Xu, C., Sun, W.: ODTC: An online darknet traffic classification model based on multimodal self-attention chaotic mapping features. Electron. Res. Arch. **31**(8), 5056–5082 (2023). https://doi.org/10.3934/era.2023259

[18] Meng, X., Srivastava, A., Arunachalam, A., Ray, A., Silva, P.H., Psiakis, R., Makris, Y., Basu, K.: NSPG: Natural language processing-based security property generator for hardware security assurance. In: Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC 2024) (2024)

[19] Lin, X., Xiong, G., Gou, G., Li, Z., Shi, J., Yu, J.: ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In: Proceedings of The Web Conference (WWW) 2022, pp. 633–642. ACM (2022)

[20] Huoh, T.-L., Luo, Y., Li, P., Zhang, T.: Flow-based graph neural network for encrypted network traffic classification using geometric deep learning. IEEE Trans. Netw. Serv. Manag. **20**(2), 1224–1237 (2023)

[21] Alwhbi, I.A., Zou, C.C., Alharbi, R.N.: Encrypted network traffic analysis and classification utilizing machine learning. Sensors (Basel) **24**(11), 3509 (2024). https://doi.org/10.3390/s24113509

[22] Houssel, P.R., Singh, P., Layeghy, S., Portmann, M.: Towards explainable network intrusion detection using large language models. In: Proceedings of the 2024 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT), pp. 67–72. IEEE/ACM (2024). https://doi.org/10.1109/BDCAT63179.2024.00021

[23] Neupane, S., Ables, J., Anderson, W., Mittal, S., Rahimi, S., Banicescu, I., Seale, M.: Explainable artificial intelligence (XAI) in intrusion detection systems: A review. J. Netw. Comput. Appl. **194**, 103217 (2022)

[24] Lee, J., Rew, J.: Vision-language model-based local interpretable model-agnostic explanations for machine learning-based CAN intrusion detection. Sensors **25**(10), 3020 (2025)

[25] El-Maghraby, R.T., Abd Elazim, N.M., Bahaa-Eldin, A.M.: Deep packet inspection techniques: Survey and challenges. Int. J. Comput. Netw. Commun. **9**(3), 35–48 (2017)